

ZJU Summer 2024 Contest 4

Contest Analysis

Group A

07.19.2025

Table of Contents

1 Reset the Circular Ring

2 Counting Stars

3 XOR Distance Sum

4 Highbit of XOR

5 The Lone Gladiator

6 Big Tree Cutting Game

Reset the Circular Ring

设 $d = \gcd(m, n)$ ，不难发现一次操作涉及恰好 n/d 个位置，以这 n/d 个位置中任意一个位置为起点，操作都是等价的。如果把这 n/d 个位置想象成一个环，操作等价于对环上的数进行了一次旋转。

进一步，我们可以把 n 个位置划分为 d 个子环，每次操作都是对子环的一次旋转。两种情况：如果对某一个子环单独进行 n/d 次旋转，环是不变的；如果对每个子环都进行一次旋转，得到的环也和原环同构。故答案为 $\min(d, n/d)$ 。

后置数论知识： $(km + i) \% n$ ($k \in \mathbb{Z}$, m, n 为常数) 随着 i 的偏移恰好能产生 $\gcd(m, n)$ 个等价类，每个等价类中恰有 $n / \gcd(m, n)$ 个数。

Table of Contents

- 1 Reset the Circular Ring
- 2 Counting Stars
- 3 XOR Distance Sum
- 4 Highbit of XOR
- 5 The Lone Gladiator
- 6 Big Tree Cutting Game

首先，我们不妨将字符集修改为 a，直接执行 `b:a c:a`
接着，我们只需模拟加法进位，注意 $9+1=10$ 这里进位的写法即可：
`0a:1 1a:2 2a:3 3a:4 4a:5 5a:6 6a:7 7a:8 8a:9 9a:a0 a:1`

Table of Contents

- 1 Reset the Circular Ring
- 2 Counting Stars
- 3 XOR Distance Sum**
- 4 Highbit of XOR
- 5 The Lone Gladiator
- 6 Big Tree Cutting Game

考虑直接点分治。对于每层，我们只需要维护一个支持全局 $+1$ ，查询全局异或和的数据结构，Trie 可以胜任之。
总复杂度 $O(n \log^2 n)$ 。存在 $O(n \log n)$ 的不基于点分治的做法，此处不表。

Table of Contents

- 1 Reset the Circular Ring
- 2 Counting Stars
- 3 XOR Distance Sum
- 4 Highbit of XOR**
- 5 The Lone Gladiator
- 6 Big Tree Cutting Game

Highbit of XOR

观察到 $len \geq 5$ 时，答案只在最高位 p 的个数为奇数的时候为 2^p 。这部分的统计是简单的。

$len \leq 4$ 时需要枚举二叉树以得到答案。

Table of Contents

- 1 Reset the Circular Ring
- 2 Counting Stars
- 3 XOR Distance Sum
- 4 Highbit of XOR
- 5 The Lone Gladiator**
- 6 Big Tree Cutting Game

The Lone Gladiator

考虑在 A 战败以后让 B 和 C 接着打，其中 B 获胜的概率为 p ，C 获胜的概率为 $1 - p$ ，直到决定胜者。

这样以来，A 获胜的概率 = $1 - \text{B 获胜的概率} - \text{C 获胜的概率}$

考虑 B 获胜的概率，发现这相当于让 A 和 C 一起打 B，不用关心 A 和 C 的胜负关系。于是问题变成了两个人玩游戏的情况。

The Lone Gladiator

设 $P(a, b, p)$ 表示一个人血量为 a ，一个人血量为 b ，前者获胜概率为 p 时，前者获胜的概率。

设 f_i 表示前者血量为 i 时的获胜概率，则 $f_0 = 0, f_{a+b} = 1$
有

$$f_i = pf_{i+1} + (1-p)f_{i-1}$$

即

$$f_{i+1} = \frac{1}{p}f_i - \frac{1-p}{p}f_{i-1}$$

那么我们可以把 f_i 表示成 $k_i f_1$ 的形式（直接求通式或者），然后根据 $f_{a+b} = 1$ 求出 f_i ，进而求出 $P(a, b, p) = f_a$

Table of Contents

- 1 Reset the Circular Ring
- 2 Counting Stars
- 3 XOR Distance Sum
- 4 Highbit of XOR
- 5 The Lone Gladiator
- 6 Big Tree Cutting Game**

Big Tree Cutting Game

首先考虑单组询问，如果 Alice 第一步做完了，游戏变成每次选择的边都要构成一条到根的路径的两个子游戏，获胜条件等价于判断这两个子游戏的 sg 值是否相等。对于每个游戏，选择一条边相当于把它分成一个每次只能取一条边的链和一个规则相同的子树。而链上相当于每次可以取一个石子的游戏，sg 值为链长 mod 2，那么有如下转移：

$$sg_o = \text{mex}\{sg_{to} \oplus (dep_o - dep_{to} - 1) \% 2\}$$

其中 to 为 o 的子树中的点。

Big Tree Cutting Game

如果一个点他有一个儿子 o 的 sg 值为 $2k$ 或 $2k+1$, 那么对于任何的 $0 \leq s \leq k$ 肯定能在这个儿子内部找到一个点使得 $sg_{to} \oplus (dep_o - dep_{to} - 1) \% 2 = s$, 因为这些值可以两两配对, 所以也一定能找到 $sg_{to} \oplus (dep_{fa_o} - dep_{to} - 1) \% 2 = sg_{to} \oplus (dep_o - dep_{to}) \% 2 = s$ 。若儿子 sg 值为 $2k+1$, 假如当前点能通过选择这个子树里的一条边到达 $2k$ 这个状态, 那么这个儿子选这条边就能到达 $2k+1$, 与其 sg 值矛盾, 同理若能到达大于等于 $2(k+1)$ 的点, 那么从这个点逐步向上转移也能推出其与儿子 sg 值矛盾。若为 $2k$ 同理。

Big Tree Cutting Game

所以说一个点的一个儿子 sg 值为 $2k/2k+1$ ，那么它能到达的后继状态就多了 0 到 $2k-1$ 和 $2k/2k+1$ 。

于是我们容易发现，一个点的 sg 值只和其儿子 sg 的最大值与次大值有关，假如两者除以 2 下取整相等，即分别为 $2k$ 与 $2k+1$ ，则该点 sg 值就为 $2(k+1)$ ，反之其为儿子 sg 最大值异或 1 。

我们下面直接定义一个点 sg 值除以 2 下取整的结果为 k ，同时令 k 相比其儿子增加了的点作为 k 级关键点。

Big Tree Cutting Game

那么一个 k 级关键点只会在其子树内没有 k 级关键点，且子树内存在两个深度奇偶不同的 $k-1$ 级关键点，然后所有节点的 sg 值都能通过其子树中最大的那个关键点以及其到当前节点的距离来确定。

同时从上述可以得知，产生一个 k 级关键点至少需要 2^k 的点，所以一条到根的路径上至多只会出现 $\log n$ 级关键点。然后每次询问中删一条边与加一条边的操作都相当于重构当前点到根的那些关键点，重构的时候暴力跳到下一个关键点的复杂度是对的。

Big Tree Cutting Game

实现这个重构的方法可能有很多，std 的实现方法是对原树重剖，对每一个重链存下其原来的关键点以及对于每个节点预处理出如果这条重链上比它低 / 高的最大关键点值为 k 、深度为奇数与偶数时下一个关键点的位置。删边相当于重构 $O(\log n)$ 个重链里总计 $O(\log n)$ 个关键点的信息，同时会让每条重链有 $O(1)$ 个因其轻儿子信息修改而无法利用预处理信息跳过的点，加边相当于从根往下跳，直到跳的位置越过了被删边修改过轻儿子的点或者出了实际换根的路径的时候再暴力在这些位置统计贡献，需要暴力统计位置的地方是 $O(\log n)$ 的，每次最多跳 $O(\log n)$ 次，所以能 $O(\log n)$ 得到答案。总时间复杂度是 $O(q \log n)$ 。